

Wspólne wejścia (dla wszystkich torów)

- LiDAR 2D: `sensor_msgs/LaserScan` (używane 360 próbek na skan)
- Odometria: `nav_msgs/Odometry` (w trybie AI dodatkowo „zazsumiona” i/lub „korygowana”)

Tor 1: SLAM Toolbox – baseline (klasyczny SLAM)

Node: `slam_toolbox_baseline` (`slam_toolbox/sync_slam_toolbox_node`, `lifecycle`)

Wejścia:

- `/scan_slam` (`LaserScan`)
- odometria/TF zgodnie z konfiguracją `slam_toolbox_baseline.yaml`

Wyjścia:

- `/map` (`OccupancyGrid`)
- TF `map -> odom` (typowe dla `slam_toolbox`)
- pozycja robota (pośrednio przez TF / internal pose)

Rola w porównaniu:

- główny klasyczny punkt odniesienia (pełny SLAM, mapa + lokalizacja)

► Kliknij, aby zobaczyć Pseudokod (Baseline Logic)

```
def run_baseline_slam():
    """
    Konfiguracja: slam_toolbox_baseline.yaml
    Node: slam_toolbox_baseline (lifecycle)
    """

    # 1. Wejście: Skaner i TF (z dryfem)
    scan = subscribe("/scan_slam") # Znormalizowany LiDAR
    base_pose = lookup_tf("odom", "base_link") # Pozycja zazsumiona (odometria)

    # 2. Parametry (slam_toolbox_baseline.yaml)
    resolution = 0.05 # [m/pixel]
    max_laser_range = 10.0 # [m]

    # 3. Karto Scan Matcher (Graph SLAM)
    # SLAM próbuje dopasować skan do mapy, startując z pozycji 'odom'
    corrected_pose = karto_match(
        scan,
        initial_guess=base_pose,
        search_window=0.5 # domyślny
    )

    # 4. Wyjście
    # Publikacja mapy i korekty TF (niwelującej dryf 'odom')
    publish_topic("/map", resolution=0.05)
    publish_tf(parent="map", child="odom")
```

Tor 2: SLAM Toolbox + AI (SLAM z korygowaną odometrią)

Node: `slam_toolbox_ai` (`slam_toolbox/sync_slam_toolbox_node`, `lifecycle`)

Wejścia:

- `/scan_slam`
- odometria „AI”: TF `/odom_ai` publikowane przez `infer_node`

Wyjścia:

- `/map_ai` (OccupancyGrid) ← (remap z `/map`)
- TF `map_ai` -> `odom_ai` (zgodnie z konfiguracją `slam_toolbox_ai`)
- `/pose_ai` (PoseStamped) z modułu AI (patrz poniżej)

Rola w porównaniu:

- główny tor „AI-SLAM” oceniany względem baseline

► Kliknij, aby zobaczyć Pseudokod (AI Pipeline Logic)

```
def run_ai_slam_pipeline():
    """
    Pipeline: infer_node -> slam_toolbox_ai
    Config: experiment_config.yaml (sekcja 'inference' i 'slam')
    """

    # --- KROK 1: Inferencja (infer_node.py) ---
    # Wejścia zdefiniowane w experiment_config.yaml
    scan_in = subscribe("/scan_slam")
    odom_in = subscribe("/odom")          # Zaszumiona odometria

    # Model MLP przewiduje błąd (dx, dy, dtheta)
    correction = model.predict(scan_in, odom_in)

    # Obliczanie "czystej" odometrii
    odom_ai_pose = odom_in.pose + correction

    # Wyjście inferencji: Nowa ramka TF
    # Służy jako "lepszy start" dla SLAM-a
    publish_tf(parent="odom_ai", child="base_link")

    # --- KROK 2: SLAM (slam_toolbox_ai) ---
    # Konfiguracja: slam_toolbox_ai.yaml

    # SLAM używa poprawionej ramki 'odom_ai' zamiast zwykłego 'odom'
    # odom_frame: "odom_ai"

    current_scan = subscribe("/scan_slam")
    guess_pose = lookup_tf("odom_ai", "base_link")

    # Budowanie mapy z mniejszym błędem wejściowym
    update_map(current_scan, guess_pose)

    # Wyjście: Mapa na osobnym topicu (z remapowania)
    publish_topic("/map_ai")
```

Tor 3: Scan Matcher – local (klasyczne dopasowanie skanów, szybkie)

Node: `scan_matcher_local` (`ai_slam_bringup/scan_matcher.py`)

Wejście:

- `scan_topic=/scan_slam`

Wyjścia:

- `pose_topic=/pose_scanmatch` (PoseStamped, frame_id="odom" – kompatybilne z ewaluacją)
- `twist_topic=/twist_scanmatch` (TwistStamped: v, omega)
- TF (opcjonalnie): `odom_scanmatch -> base_link_scanmatch`

Opis działania:

- estymacja ruchu między kolejnymi skanami (dx, dy, dθ)
- metoda „local”: wielopoziomowe przeszukiwanie małego okna (szybka)

Najważniejsze parametry:

- `grid_res, grid_extent, max_use_range, max_points`
- okna i kroki local search: `local_lvl1_*`, `local_lvl2_*`, `local_lvl3_*`

►  **Kliknij, aby zobaczyć Pseudokod (Local Search)**

```
def run_scan_matcher_local():
    """
    Node: scan_matcher_local
    Param: method="local"
    """
    prev_scan = None
    global_pose = (0, 0, 0) # Start w (0,0,0)

    while True:
        curr_scan = subscribe("/scan_slam")

        # Algorytm Local Search (Gradient / Hill Climbing)
        # Szuka tylko w bardzo małym otoczeniu poprzedniej pozycji
        dx, dy, dth = find_match_local(
            target=prev_scan,
            source=curr_scan,
            search_step=0.01 # Precyzyjny, mały krok
        )

        # Integracja wyniku (Dead Reckoning)
        global_pose += (dx, dy, dth)

        # Wyjścia (zgodne z demo.launch.py)
        publish("/pose_scanmatch", global_pose) # frame_id="odom"
        publish("/twist_scanmatch", (dx/dt, dth/dt))

        prev_scan = curr_scan
```

Tor 4: Scan Matcher – bruteforce (klasyczne dopasowanie skanów, referencja)

Node: `scan_matcher_bruteforce` (`ai_slam_bringup/scan_matcher.py`)

Wejście:

- `scan_topic=/scan_slam`

Wyjścia:

- `pose_topic=/pose_bruteforce`
- `twist_topic=/twist_bruteforce`
- TF (opcjonalnie): `odom_bruteforce -> base_link_bruteforce`

Opis działania:

- pełne przeszukiwanie zakresów ($dx, dy, d\theta$); wolniejsze, ale stabilne jako referencja
- zwykle publikowane rzadziej parametrem `publish_every_n`

Najważniejsze parametry:

- `bf_range_xy`, `bf_range_th`, `bf_step_xy`, `bf_step_th`
- `publish_every_n`

►  **Kliknij, aby zobaczyć Pseudokod (Bruteforce Grid Search)**

```
def run_scan_matcher_bruteforce():
    """
    Node: scan_matcher_bruteforce
    Param: method="bruteforce"
    Parametry z demo.launch.py (konfigurowalne)
    """
    # Zakresy przeszukiwania siatki (Grid Search)
    LIMIT_XY = 0.15 # parametr: bf_range_xy
    STEP_XY = 0.01 # parametr: bf_step_xy
    LIMIT_TH = 0.25 # parametr: bf_range_th (rad)

    # Pętle po wszystkich możliwych kombinacjach przesunięcia
    # (To obciąża CPU, dlatego publish_every_n=5)
    best_score = -1
    best_transform = (0,0,0)

    for x in range(-LIMIT_XY, LIMIT_XY, STEP_XY):
        for y in range(-LIMIT_XY, LIMIT_XY, STEP_XY):
            for th in range(-LIMIT_TH, LIMIT_TH, 0.01):

                score = check_overlap(curr_scan, prev_scan, offset=(x,y,th))

                if score > best_score:
                    best_score = score
                    best_transform = (x, y, th)

    # Wyjście
    publish("/pose_bruteforce", integrate(best_transform))
```

Tabela porównawcza 4 torów estymacji (wejścia/wyjścia + znaczenie)

Legenda: (x, y, θ) = pozycja i orientacja robota; (Δx , Δy , $\Delta \theta$) = przyrost ruchu między kolejnymi skanami LiDAR.

Tor	Metoda (idea)	Node / implementacja	Wejścia (ROS)	Wyjścia (ROS)	Co oznaczają wyjścia (semantyka)
1. Baseline SLAM	Klasyczny SLAM (mapowanie + lokalizacja)	<code>slam_toolbox_baseline</code> (<code>slam_toolbox/sync_slam_toolbox_node</code>)	<code>/scan_slam</code> + odometria/TF (dryf)	<code>/map</code> + TF <code>map -> odom</code>	Mapa otoczenia + globalna trajektoria robota w układzie mapy (pośrednio przez TF / wewn. estymator).
2. AI SLAM	SLAM Toolbox z odometrią korygowaną przez AI	<code>slam_toolbox_ai</code> + <code>infer_node</code>	<code>/scan_slam</code> + odometria „AI” (<code>odom_ai</code> /TF z inferencji)	<code>/map_ai</code> + <code>/pose_ai</code> + TF (dla toru AI)	<code>/pose_ai</code> to korygowana pozycja (x,y,θ) . W tle AI estymuje korekcję ruchu (w praktyce odpowiadającą ($\Delta x, \Delta y, \Delta \theta$)), integruje ją do pozycji i publikuje „lepszą” odometrię dla SLAM.
3. Scan-to-scan (local)	Estymacja ruchu między skanami (szybka, lokalna)	<code>scan_matcher_local</code> (<code>scan_matcher.py</code>)	<code>/scan_slam</code>	<code>pose_topic=/pose_scanmatch</code> , <code>twist_topic=/twist_scanmatch</code>	Algorytm liczy ruch między skanami : ($\Delta x, \Delta y, \Delta \theta$), a następnie integruje to do pozycji (x,y,θ) publikowanej w <code>/pose_scanmatch</code> . Dodatkowo <code>/twist_scanmatch</code> to prędkości: v i ω wyliczone z tych przyrostów.
4. Scan-to-scan (bruteforce)	Klasyczne dopasowanie skanów przez przeszukiwanie siatki (wolniejsze, referencyjne)	<code>scan_matcher_bruteforce</code> (<code>scan_matcher.py</code>)	<code>/scan_slam</code>	<code>pose_topic=/pose_bruteforce</code> , <code>twist_topic=/twist_bruteforce</code>	Analogicznie: w środku powstaje ($\Delta x, \Delta y, \Delta \theta$) z przeszukiwania, a <code>/pose_bruteforce</code> to zintegrowana pozycja (x,y,θ) . <code>/twist_bruteforce</code> = v i ω z przyrostów.